

Data Collection & Cleaning Part –2

```
import pandas as pd

data = {
    "Salary": [25000, 28000, 30000, 32000, 35000, 38000, 40000, 45000,
500000]
}

df = pd.DataFrame(data)
print(df)
```

This creates a sample dataset containing employee salaries. The value **500000** appears much larger than the other values and may be an outlier.

Visualize Data Using Boxplot

```
import matplotlib.pyplot as plt

plt.boxplot(df["Salary"])
plt.show()
```

This creates a boxplot to visually identify outliers in the dataset.

Calculate Quartiles

```
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)

print("Q1:", Q1)
print("Q3:", Q3)
```

This calculates the first quartile (Q1) and third quartile (Q3) of the salary data.

Calculate IQR

```
IQR = Q3 - Q1

print("IQR:", IQR)
```

This calculates the Interquartile Range (IQR), which measures the spread of the middle 50% of the data.

Define Outlier Limits

```
lower_limit = Q1 - (1.5 * IQR)
upper_limit = Q3 + (1.5 * IQR)

print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
```

This calculates the lower and upper boundaries used to detect outliers.

Detect Outliers

```
outliers = df[
    (df["Salary"] < lower_limit) |
    (df["Salary"] > upper_limit)
]

print(outliers)
```

This finds all salary values that fall outside the valid range.

Remove Outliers

```
df_no_outliers = df[
    (df["Salary"] >= lower_limit) &
    (df["Salary"] <= upper_limit)
]

print(df_no_outliers)
```

This removes rows containing outlier values from the dataset.

Cap Outliers

```
df["Salary"] = df["Salary"].clip(
    lower=lower_limit,
    upper=upper_limit
)
```

```
print(df)
```

This replaces extreme values with the calculated limits instead of removing them.

Data Type & Format Fixing

Create Sample Dataset

```
import pandas as pd

data = {
    "Name": [" alice ", "BOB", "Charlie"],
    "Age": ["25", "30", "35"],
    "Salary": ["50000", "60000", "70000"],
    "Join_Date": ["01-01-2024", "15-02-2024", "20-03-2024"]
}

df = pd.DataFrame(data)

print(df)
```

This creates a dataset with inconsistent text formatting and incorrect data types.

Check Current Data Types

```
print(df.dtypes)
```

This displays the current data type of each column.

Convert Age Column to Integer

```
df["Age"] = df["Age"].astype(int)

print(df.dtypes)
```

This converts the Age column from string to integer.

Convert Salary Column to Float

```
df["Salary"] = df["Salary"].astype(float)
```

This converts the Salary column into a numeric format.

Convert Date Column to Datetime

```
df["Join_Date"] = pd.to_datetime(df["Join_Date"])
```

This converts date values into pandas datetime format.

Remove Extra Spaces

```
df["Name"] = df["Name"].str.strip()  
  
print(df["Name"])
```

This removes unwanted spaces from the beginning and end of text values.

Convert Text to Lowercase

```
df["Name"] = df["Name"].str.lower()
```

This converts all names to lowercase for consistency.

Convert Text to Uppercase

```
df["Name"] = df["Name"].str.upper()
```

This converts all names to uppercase.

Convert Text to Title Case

```
df["Name"] = df["Name"].str.title()
```

This converts the first letter of each word to uppercase.

Replace Incorrect Values

```
df["City"] = ["NY", "New York", "new york"]

df["City"] = df["City"].replace({
    "NY": "New York",
    "new york": "New York"
})

print(df["City"])
```

This standardizes different versions of the same value.

Rename Column Names

```
df.columns = [
    "Employee_Name",
    "Employee_Age",
    "Employee_Salary",
    "Join_Date",
    "City"
]
```

This changes column names to more meaningful names.

Verify Final Data Types

```
print(df.dtypes)
```

This checks the final data types after all conversions.

Problem S